

## Functional Programming

- Lambda Expressions
- Stream Programming

## File IO

## Multi-Threading

## Annotation & Reflection

- Enum

## Packages, GC, Collection

- Revision

## Function Interface

- an interface that has SAM is called as Functional Interface
- It provides a method design that needs to be implemented
- Comparator is a functional interface
- `int compare(T e1,T e2)->e1.id-e2.id`

## # Method References

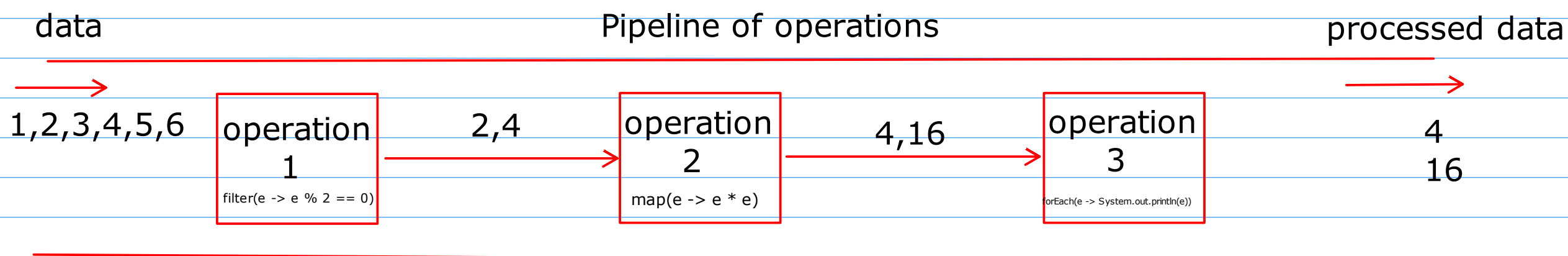
- Shorthand Implementation of Lambda Expressions

```
Integer []arr = {1,2,3,4,5};
```

1. get even number
2. square the even number
2. display square of even numbers

```
for(Integer e:arr){  
    if(e %2 == 0){  
        int sqr = e * e  
        println(sqr)  
    }  
}
```

## Stream Programming



## Stream Programming

### Stream Operations are of 2 types

#### 1. Intermediate operation

- Any operation that creates another stream object is called as intermediate operation
- Can be Stateful or Stateless operations

#### 2. Terminal Operation

- Any Operation that returns the result or void is called as Terminal Operation

## # Characteristics of Stream

1. Streams can never be processed if terminal operations are not given  
i.e Streams are lazy evaluated
2. Once Stream is processed it cannot be used again
3. Every intermediate operation on Stream will generate a new Stream object
- 4.